ID	Type (F/E)	Feature	User / Technical Story	Acceptance Criteria	Story Points	Jira Key	GitHub
Sprint 1 (22 Sep to 19 Oct): Data, model and app					**79**		
US_01	**E**	Data Pipeline	As the development team, we need clean per-eye labels, a patient-level split and preprocessed photos, so the model can be trained and tested fairly.	Per-eye labels taken from each eye's own keywords pass the verification script; patients are split 70/15/15 with seed 42; one shared preprocessing function runs on every photo; code is merged into main.	21	SCRUM-142	scripts/verify_labels.py | PRs 15, 16
US_02	**E**	Dual-Head Model	As the ML team, we need one shared ResNet with a cataract head and a glaucoma head trained on the ODIR-5K training split, so that one photo can later give both results in the app.	A shared ResNet with two heads trains end to end on the real labels, weights rare cases, never reads external data, and passes the smoke test.	13	SCRUM-143	scripts/train_dual.py | PRs 27, 28, 29
US_03	**E**	External Test Sets	As the ML team, we need test-only labels for four outside datasets and a check that none of their photos are in ODIR-5K, so that the model is tested on data it never saw.	Test-only labels for ORIGA, DRISHTI-GS, ACRIMA and retina_dataset_2016 have checked counts; the overlap check passes; the metrics protocol is written; calibration code is ready and uses the validation split only.	13	SCRUM-144	labels/external/ | PR 26
US_04	**F**	App Prototype (carried over to Sprint 2)	As a screening technician, I want to upload a patient's fundus photo and see a clear cataract and glaucoma result, so I can decide quickly whether the patient should see an eye specialist.	Upload, Results and History screens work; upload shows a preview and rejects non-images; results show separate cataract and glaucoma results from the API, with "Uncertain, refer" as a normal result; mock data only; follows the DUAL branding.	21	SCRUM-31	frontend/ | PRs 35, 36, 37, 57
US_05	**F**	Photo Quality Check	As a screening technician, I want the app to stop photos that can't be screened and tell me why, so I don't get a result from an unusable file.	Unreadable, almost-black and non-fundus uploads are blocked with a clear message; hazy or blurry eye photos still go to the model; all ODIR training photos pass; 17 automated tests pass.	3	SCRUM-127	scripts/quality_check.py | PRs 30, 31
US_06	**E**	Model Ablation	As the development team, we want cataract-only, glaucoma-only and shared two-head models trained with identical settings, so we can test whether sharing the backbone helps.	Nine models are saved (3 model types x 3 seeds); the training split is used to learn and the validation split to stop; settings for every run are recorded; no external data is used.	8	SCRUM-55	results/task11/ | PRs 33, 34, 41, 42, 46
Sprint 2 (20 Oct to 16 Nov)					**44**		
US_07	**F**	Calibration & Abstention	As an optometrist, I want the tool to say "low image quality or uncertain, refer" when it is not sure, so that I do not act on a weak result.	Calibration, one decision threshold per disease and one abstention cutoff are fitted on the validation split only; the frozen values are committed and not changed afterwards.	5	SCRUM-56	results/calibration/ | PRs 43, 54, 73
US_08	**E**	Internal Test	As the development team, we want a within-dataset baseline, so that external results can be compared against it.	All three model types are scored once on the internal test split after calibration is frozen; AUC with 95% confidence interval for each disease; results table is in the repository.	3	SCRUM-57	results/task13/ | PRs 74, 76, 77, 79, 83
US_09	**E**	External Testing	As the ML team, we need the frozen models scored on ORIGA, DRISHTI-GS, ACRIMA and retina_dataset_2016, so that we can report how well the model works on photos from other clinics.	ORIGA and DRISHTI-GS are scored with the frozen thresholds and calibration; ACRIMA and retina_dataset_2016 are reported in their own rows; AUC with confidence intervals and calibration for each set.	5	SCRUM-58	results/external/ | PRs 81, 82, 87
US_10	**E**	Technical Paper Draft	As the team writing the technical paper, we need the dataset and limitations sections written with verified counts, so that readers can judge the results.	Dataset and limitations sections use counts from the verification script; the cataract external-testing gap is stated as a limitation; PDF and Word versions are on the wiki.	5	SCRUM-59	Not on GitHub yet
US_11	**E**	Cataract Haze Check	As the ML team, we need to check whether the cataract head reacts to blurry or hazy photos rather than the lens, so that a cataract referral can be trusted.	The outcome that means "the model detects haze" is written down before results are seen; cataract share is reported for media-opacity and normal eyes; blur curve is plotted against real cataract eyes.	5	SCRUM-61	results/task15/ | PRs 70, 85, 86, 88, 89, 91
US_16	**E**	Testing and Review	As the team, we want every Sprint 2 story tested and reviewed against its acceptance criteria by someone who did not build it, so that we know each story really works before we close it.	Test cases are written for each Sprint 2 story; each is run and recorded as pass or fail with the date; every pull request for a Sprint 2 story is reviewed before it is merged; problems found are logged in Jira and linked to the story.	5	SCRUM-154	documents/test_cases/

ID	Type (F/E)	Feature	User / Technical Story	Acceptance Criteria	Story Points	Jira Key	GitHub
US_18	**E**	Cloud Back End	As the back-end developer, I need failed uploads cleaned up and deletions kept in step between the database and storage, so that one failed upload never blocks the app.	A failed upload leaves no leftover file, and the next upload works. Deleting a photo never leaves the database and storage out of sync. The API tests run against a separate test database (never the shared one) and pass, including the PDF exports. Anyone can set up the back end from the repo using the schema, .env.example and README. The docs and diagrams describe Supabase, not SQLite.	8	SCRUM-157	backend/ | PRs 90, 93
US_12	**E**	Glaucoma Heatmap Check	As the ML team, we need Grad-CAM heatmaps checked against the optic disc outlines on ORIGA and DRISHTI-GS, so that we can show a glaucoma result is based on the disc.	Grad-CAM heatmaps are produced for ORIGA and DRISHTI-GS; the share of images whose heatmap peak falls inside the disc outline is reported per dataset.	5	SCRUM-62	results/task16/ | PRs 92, 94
US_13	**E**	Ablation Comparison	As the ML team, we need the single-disease and shared models compared per disease, so that the paper can justify one shared backbone.	Single-disease and shared models are compared per disease on the internal test and, for glaucoma, on the external tests; spread across seeds is shown; table and conclusion are in the repository.	3	SCRUM-63	PR 95 (open)
Sprint 3 (planned)					**34**		
US_14	**F**	Real Model in App	As an optometrist, I want to upload a fundus photo and get real cataract and glaucoma risk scores, so that I can decide whether to refer.	The app runs on the trained model; "Uncertain, refer" is shown as a normal result; every results table names the commit that produced it; the results commit is tagged before the final demo.	8	SCRUM-64	Not on GitHub yet
US_15	**E**	Final Documentation	As a developer joining later, I want install, deployment and API manuals, so that I can run and extend the app.	Data card, user manual, deployment manual and API documentation in PDF and Word; final technical paper with its history page; reviewed by the data pipeline and external validation roles.	8	SCRUM-65	Not on GitHub yet
US_17	**E**	Testing and Review	As the team, we want every Sprint 3 story tested and reviewed against its acceptance criteria by someone who did not build it, so that the final product we demo and hand over really works.	Test cases are written for each Sprint 3 story; each is run and recorded as pass or fail with the date; the full app is tested end to end with the real model before the final demo; every pull request is reviewed before it is merged; problems found are logged in Jira.	5	SCRUM-156	Not on GitHub yet
US_19	**F**	Dashboard and Patients	As a screening technician, I want a dashboard of recent screenings and referrals, a patient directory I can search, and a form to add a new patient, so that I can register patients and find their screenings quickly.	Dashboard shows the number of screenings and referrals and the most recent screenings; patient directory lists every patient, can be searched by name or patient ID, and opens a patient's past screenings; add new patient form checks required fields, saves the patient, and the new patient appears in the directory; data comes from the live back end (API and database changes included in this story); made-up patient data only; pages need sign-in and reload without errors; screens work in both themes and on narrow screens; frontend test cases are written and run.	8	SCRUM-164	Not on GitHub yet
US_20	**E**	MVP Demo	As the course instructor, I want a live demo of the finished MVP working end to end, so that I can check the product against its goals and give final feedback.	Live demo on the deployed app, not screenshots: sign in, add a patient, upload a fundus photo, see both results with the referral advice, and find the screening again in the patient directory; latest test results are shown, including any failures; final model metrics (internal and external) are reported; a backup recording of the same flow is ready in case the live app is down.	5	SCRUM-66	Not on GitHub yet
					Total **157**		

Type: F = functional story (a feature a specific user sees or uses in the app); E = enabler (technical work that supports a feature: data, model training, evaluation, documentation, testing). GitHub: link to the work on the main branch; PR numbers refer to github.com/htmw/F2026-EyeQ/pull/<number>.

Story points use the Fibonacci scale (1, 2, 3, 5, 8, 13, 21, 34). Sprint 1 stories were re-estimated on 9 Nov 2026: US_01 23 to 21, US_02 16 to 13, US_03 15 to 13, US_04 18 to 21 (US_05 3 and US_06 8 unchanged).